Independent University, Bangladesh

CSE313 Compiler Construction · Practice Set B · Mid + Quiz · Time: 1h30m

New questions. Q2 needs a conversion before LL(1); quiz Q2 has hidden left recursion.

Question paper

Mid Question (Total marks: 20)

1. Prove whether the following grammar is Ambiguous or NOT. If the grammar is ambiguous, convert it to unambiguous form. **[7]**

```
A → A + A | B
B → B * B | C
C → ( A ) | x
```

2. The grammar below is left recursive. Make it ready for LL(1), then write First() and Follow(), draw the LL(1) Parsing Table and parse `n - ( n )`. (b) Parse `n ( n`. If parsing is not possible, point out the reasons. **[8]**

```
E → E - T | T
T → n | ( E )
```

3. Find True/False: **[5]**

(a) Left factoring removes left recursion.
(b) An ambiguous grammar can be LL(1).
(c) For A → ε, the rule is placed in M[A, b] for every b in FOLLOW(A).
(d) The lexical analyzer produces the parse tree.
(e) In LL(1), the first L means the input is scanned Left to right.

Quiz Question (Total marks: 10)

1. Convert the following Non-Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[5]**

```
A → a b B | a B | c d g | c d e B | c d f B
B → x
```

2. Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[5]**

```
S → A a | b
A → A c | S d | e
```

Answers (exam layout, with comments)

Answer 1 [7]

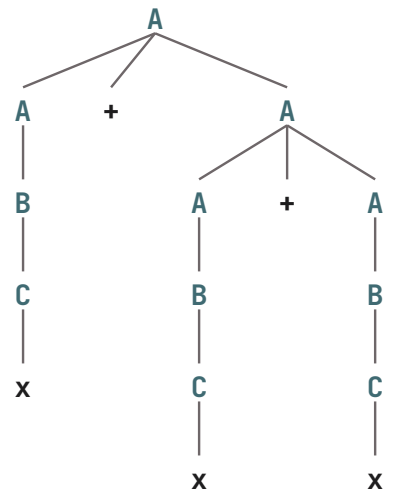
`WRITE THIS`

Yes, the grammar is ambiguous. `A → A + A` and `B → B * B` both have the same non-terminal on both sides. String: `x + x + x`

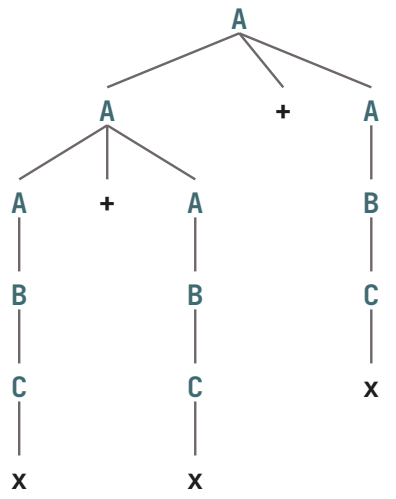
LMD 1 (right + done first): A ⇒ A + A ⇒ B + A ⇒ C + A ⇒ x + A ⇒ x + A + A ⇒ x + B + A ⇒ x + C + A ⇒ x + x + A ⇒ x + x + B ⇒ x + x + C ⇒ x + x + x

LMD 2 (left + done first): A ⇒ A + A ⇒ A + A + A ⇒ B + A + A ⇒ C + A + A ⇒ x + A + A ⇒ x + B + A ⇒ x + C + A ⇒ x + x + A ⇒ x + x + B ⇒ x + x + C ⇒ x + x + x

Tree 1



Tree 2



Since for the string `x + x + x` there exists more than one leftmost derivation parse tree, the grammar is ambiguous. (`x * x * x` also has two trees.)

[Assuming + and * are both left associative; * has higher precedence, as the levels already show]

Unambiguous grammar: make each rule recurse on one side only.

```
A → A + B | B
B → B * C | C
C → ( A ) | x
```

Operator	Precedence	Associativity
+	lowest	left (A → A + B)
*	highest	left (B → B * C)

Checked by computer: same 60 strings up to length 7, each with one LMD.

> **Lose marks by:** fixing only one of the two levels; picking x + x * x (it has only one tree here).

Answer 2 [8]

WRITE THIS

Step 0 · Remove left recursion

E → E - T | T: α = `- T`, β = `T`.

```
E → T E'
E' → - T E' | ε
```

Final RRG:

```
E → T E'
E' → - T E' | ε
T → n | ( E )
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

T → n | (E) start with different terminals: no left factoring needed.

Numbered rules: 1. E → T E' 2. E' → - T E' 3. E' → ε 4. T → n 5. T → (E)

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
E	{ n, (}	{ $,) }
E'	{ -, ε }	{ $,) }
T	{ n, (}	{ -, $,) }

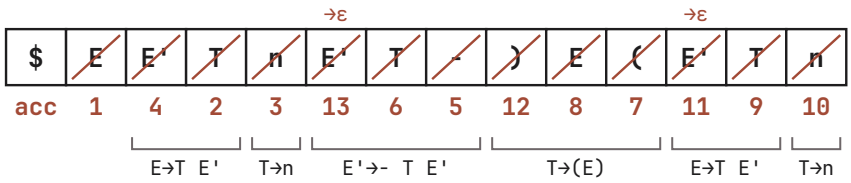
Step 2 · LL(1) parsing table

	-	n	(	)	$
E		E→T E'	E→T E'		
E'	E'→- T E'			E'→ε	E'→ε
T		T→n	T→(E)		

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3a · Parse `n - ( n )` (horizontal stack)

Buffer: `n - ( n ) $` LA ↑ starts at `n`



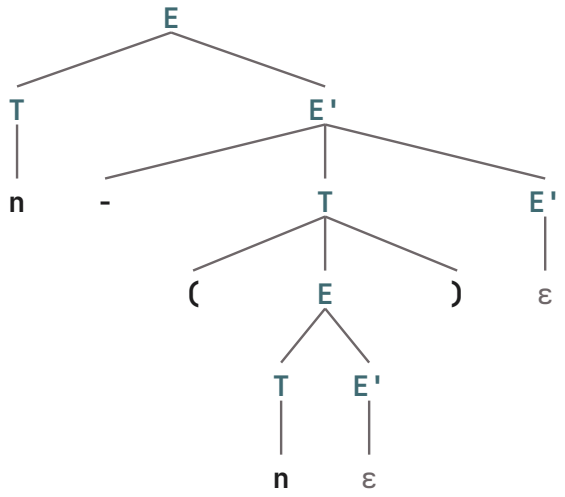
How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop E (LA = n): E→T E', push E' T
2. pop T (LA = n): T→n, push n
3. match n, pop it, move LA → -
4. pop E' (LA = -): E'→- T E', push E' T -
5. match -, pop it, move LA → (
6. pop T (LA = (): T→(E), push) E (
7. match (, pop it, move LA → n
8. pop E (LA = n): E→T E', push E' T
9. pop T (LA = n): T→n, push n
10. match n, pop it, move LA →)
11. pop E' (LA =)): E'→ε, push nothing
12. match), pop it, move LA → $
13. pop E' (LA = $): E'→ε, push nothing

acc. Top = $ and LA = $ → accept

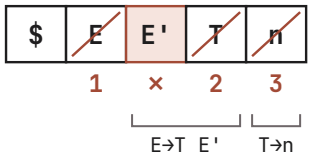
Parse tree



Leaves left to right (skip ε): `n - ( n )` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

Step 3b · Parse `n ( n` (horizontal stack)

Buffer: `n ( n $` LA ↑ starts at `n`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop E (LA = n): E→T E', push E' T
2. pop T (LA = n): T→n, push n
3. match n, pop it, move LA → (

err. Top = E', LA = (: cell M[E', (] is empty → error

Parsing is not possible. Reason: Top = E', LA = (: cell M[E', (] is empty → error. ∴ error, the string `n ( n` is not in the language.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(E): E → T E' gives { n, (}. Total { n, (}.
- FIRST(E'): E' → - T E' gives { - }. E' → ε gives ε. Total { -, ε }.
- FIRST(T): T → n gives { n }. T → (E) gives { (}. Total { n, (}.
- FOLLOW(E): start symbol → add $. in T → (E): after E is) → add {) }. Total { $,) }.
- FOLLOW(E'): in E → T E': nothing after E' → add FOLLOW(E). Total { $,) }.
- FOLLOW(T): in E → T E': after T is E' → add { - }; E' can be ε → add FOLLOW(E). in E' → - T E': after T is E' → add { - }; E' can be ε → add FOLLOW(E'). Total { -, $,) }.

Lose marks by: building the table from the left recursive grammar; FOLLOW(E') must include) (from (E)).

Answer 3 [5]

WRITE THIS

#	Answer	Reason (one line)
(a)	**False**	Left factoring removes common prefixes (NDG → DG). Left recursion needs A → βA', A' → αA' | ε.
(b)	**False**	Two trees for a string means two rules compete for some cell.
(c)	**True**	ε rules go under the FOLLOW terminals of the left side.
(d)	**False**	The lexer makes tokens; the syntax analyzer (parser) makes the parse tree.
(e)	**True**	L = left-to-right scan, L = leftmost derivation, 1 = one look-ahead.

Quiz Answer 1 [5]

WRITE THIS

Round 1: in A, `a b B`, `a B` all start with `a`. Take it out once:

```
A → a A' | c d g | c d e B | c d f B
A' → b B | B
```

Round 2: in A, `c d g`, `c d e B`, `c d f B` all start with `c d`. Take it out once:

```
A → a A' | c d A''
A'' → g | e B | f B
```

Final DG:

```
A → a A' | c d A''
A' → b B | B
A'' → g | e B | f B
B → x
```

Quiz Answer 2 [5]

WRITE THIS

A has an alternative that starts with S (already fixed), so hidden left recursion can come through it. Substitute S's current rules:

```
A → A c | A a d | b d | e
```

A → A c | A a d | b d | e: α = `c` , `a d`, β = `b d` , `e` .

```
A → b d A' | e A'
A' → c A' | a d A' | ε
```

Final RRG:

```
S → A a | b
A → b d A' | e A'
A' → c A' | a d A' | ε
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

> **Lose marks by:** missing the hidden recursion A → S d → A a d; substitute S into A first.